# STITCH, Replicated: From Single-Run Illustrations to $N$=3 Evidence and Cost-Controlled Semantic Escalation

Journal Extension — Reference Draft v2

Karthikeya Patana, Yashwant Rajesh, Aathira Sunil, Durga Saranyu
Sri Parameswaran, Kamesh Chandrasekar, Paresh Saxena, Soumya Joshi

**Abstract**

The conference version of STITCH characterised its deterministic semantic interpreter through four outcome regimes—enabling, protective, harmful, and irrelevant—each demonstrated by a single module/model pair evaluated once. That evidence was illustrative, and the paper said so. This extension replaces single-run illustration with $N$=3 replication and reports what survives it: two interpreter illustrations replicate cleanly and anchor the mechanism, a novel input/output reachability asymmetry explains *which* defect classes the interpreter can act on, and a new conditional-escalation architecture delivers the interpreter's benefit at 35–45% lower tool-invocation cost with no pass-rate regression on replicating cells. We migrate to an external benchmark (RTLLM v2.1) and conduct a controlled fine-tuning twin comparison between RTLCoder-Deepseek-v1.1 and its un-fine-tuned base, isolating the fine-tuning effect from prompt-format effects by running both as measured variables. Along the way we harden the evaluation harness, correcting three silent defects that had been quietly perturbing measurements. The contribution of the extension is not that STITCH works in more places, but that we now know *when* it works, *how much* it helps, and *at what cost*, on replicated rather than single-draw evidence.

## 1 Why This Extension Exists: N=1 Is Not Enough

The conference paper's Table of interpreter outcomes was built on *single runs*: one module, one model, one draw per condition. Its own Limitations section is explicit that the reported percentages are point estimates rather than measured effects. For a repair mechanism evaluated on a stochastic generator, that gap matters more than it first appears. A model at low temperature with no fixed seed can pass one draw and fail the next from sampling alone; a single-run cell that shows the interpreter "winning" cannot, on its own, distinguish a real mechanism from that variance.

This extension re-runs the mechanism under replication and reorganises the evidence around a single discipline: *no interpreter claim is made without $N \geq 3$ replication and a* `semantic_fired` *check reported in the same cell it explains.* Section 2 presents the cases that survive this bar—the places STITCH genuinely works. Section 3 presents what replication retired, because the retirements define the mechanism's scope as sharply as the successes define its power. Section 4 introduces the cost-controlled escalation architecture and its replicated results. Section 5 reports the harness corrections. Section 6 previews external-benchmark migration. Section 7 presents the fine-tuning twin comparison.

## 2 Where STITCH Works: Replicated Cases

We report five cases where STITCH's mechanism produces a genuine, replicated benefit. Two are the interpreter proper on proven illustrations; one is a mechanistic reachability result; two are the escalation architecture on cells that survive replication. Every case here clears the $N$=3 bar and, where the interpreter is involved, a `semantic_fired` check.

### 2.1 Case 1–2: Two Proven Interpreter Illustrations

The gated-input `UNUSEDSIGNAL` mechanism—the conference paper's cleanest illustration—replicates 3/3 on two modules and two models (Table 1). In `decoder_3to8` and `mux4_oe` the model omits a declared gating port on iteration one, Verilator fires `UNUSEDSIGNAL`, and the interpreter's targeted directive resolves the module in two iterations against a six-iteration failure for both controls. In every cell the `raw_adv` control tracks `baseline` exactly, isolating the effect to *interpretation*, not tool presence.

Table 1: Two proven illustrations, $N$=3 at $T$=0.1. `raw_adv` matches `baseline` in every cell, isolating the effect to interpretation.

| Module | Model | baseline | raw_adv | sem_adv |
|---|---|---|---|---|
| decoder_3to8 | gemma3:4b | FAIL — 6 | FAIL — 6 | **PASS — 2** |
| decoder_3to8 | llama3.1 | PASS — 2 | PASS — 2 | PASS — 2 |
| mux4_oe | gemma3:4b | FAIL — 6 | FAIL — 6 | **PASS — 2** |
| mux4_oe | llama3.1 | PASS — 4 | PASS — 4 | PASS — 2 |

On the enabling cells the interpreter also cuts token cost sharply—roughly 5–7× against both controls—because it converts a six-iteration failure into a two-iteration pass.

### 2.2 Case 3: The Input/Output Reachability Asymmetry

A new mechanistic result explains *which* defect classes the interpreter can reach. A forgotten gating *input* reliably produces the target defect and reaches the interpreter; a forgotten *output* does not, because the model assigns declared outputs unprompted on iteration one (Table 2).

Table 2: Forgotten inputs are reachable; forgotten outputs are not. `sf` = `semantic_fired`. The interpreter fires only where the model actually produces the target defect.

| Target class | Module | Iteration-1 behaviour | sf |
|---|---|---|---|
| Forgotten *input* | decoder_3to8 | port declared, never read | 1 |
| Forgotten *input* | mux4_oe | `oe` never referenced | 1 |
| Forgotten *output* | adder_with_overflow | output assigned unprompted | 0 |
| Forgotten *output* | fsm_next_state_error | output assigned on all paths | 0 |

The explanation is a completeness asymmetry in autoregressive generation: assigning a declared output is a *local* obligation (visibly unfinished if omitted), whereas consulting a declared input is a *non-local* one (the module reads as complete without it). The practical consequence is a sharper scope claim than the conference paper could make: reachability is a joint property of the tool and the generator's failure distribution and must be measured per class. `UNUSEDSIGNAL` on a gating input is not an arbitrary choice among handled codes; it is the class where tool visibility and real model failure are *known* to overlap.

### 2.3 Case 4–5: Escalation Wins on Replicating Cells

The conditional-escalation architecture (Section 4) produces two genuine, replicated wins on the Phase 1 benchmark (Table 3). On `pwm_generator`—the hardest cell in the set—escalation passes 2/3 where always-on `sem_adv` passes 1/3 and `baseline` passes 0/3: an outright win. On `ripple_carry_adder` escalation wins *structurally*, matching `baseline`'s clean 3/3 recovery precisely by *not* engaging tools that drive always-on `sem_adv` into a 0/3 collapse.

Table 3: The four Phase 1 cells with genuine dynamics, reported at equal weight (including the one loss). F$k$ = failure at cap.

| Cell | base | sem_adv | sem_esc | note |
|---|---|---|---|---|
| pwm_generator/deepseek | 0/3 | 1/3 | **2/3** | win |
| ripple_carry_adder/deepseek | 3/3 | 0/3 | **3/3** | structural win |
| counter_4bit/deepseek | 3/3 | 3/3 | 3/3 | cost, no fail |
| alu_8bit/gemma3:4b | 3/3 | 3/3 | 2/3 | one loss (flagged) |

We report the full four-cell picture, not a curated subset. Escalation is therefore *not* a broad pass-rate victory over always-on interpretation, and we do not claim one. Its value is on a different axis, quantified in Section 4.1.

## 3 What Replication Retired, and Why It Matters

Three larger-module candidates built to test whether the proven mechanism scales all failed $N$=3 replication. The failures define the mechanism's boundary.

**Resampling variance can masquerade as a win.** `datapath_mux_alu`/llama3.1 passed `sem_adv` in 2 of 3 samples with `semantic_fired`=0 in all three—the interpreter never fired. As a single-run cell this reads as a clean win; it is sampling variance.

**A correct directive does not guarantee recovery.** `instr_decode_unit`/gemma3:4b fired `UNUSEDSIGNAL` correctly and still failed, regressing into an unrelated degenerate syntax loop the interpreter has no vocabulary to reach.

## 4 Cost-Controlled Semantic Escalation

The conference paper's own *harmful* and *protective* regimes point to one structural weakness in firing the interpreter on every iteration: it intervenes when intervention is not needed, occasionally derailing a converging run. The escalation architecture fires the interpreter only once a run is demonstrably stuck.

`sem_escalate` starts in a baseline state, tracks consecutive failures carrying the same normalised error signature, and after two identical-signature failures escalates permanently to the interpreter path. `sem_escalate_restart` adds a blank-slate rewrite if a run stays stuck even after escalation.

### 4.1 The Cost Result

The defensible headline is efficiency (Table 4). Escalation invokes the heavyweight tools roughly 35–45% less often than always-on `sem_adv`/`raw_adv`, while matching or exceeding their pass rate. We report both cost axes and note they diverge: escalation reduces *tool invocations* but can *increase*

tokens on cells where it escalates and then runs extra iterations. The efficiency claim is scoped to the tool-invocation axis.

Table 4: Cost per run, all 255 Phase 1 runs. Efficiency claim scoped to the heavyweight-tool axis; token cost reported and moves the other way.

| **Condition** | **mean tokens/run** | **Verilator+Yosys calls/run** |
|---|---|---|
| baseline | 4891 | 0.00 |
| raw_adv | 5354 | 2.61 |
| sem_adv | 4883 | 2.37 |
| **sem_escalate** | 5151 | **1.53** |
| **sem_esc_restart** | 4719 | **1.37** |

### 4.2 Replication Collapse in the Benchmark Premise

The Phase 1 benchmark also produced a methodological result: the run set was derived from historical $N$=1 labels, and re-measured at $N$=3, 4 of 5 "real-test" and 2 of 3 "enabling" cells do not reproduce the baseline behaviour their category was defined by. This is concrete evidence that single-run RTL-repair benchmarks can encode sampling noise as apparent structure. The regression gate nonetheless holds: both proven illustrations pass 4/4 on strict pass/fail under the new conditions.

**[FIGURE PLACEHOLDER]**

Phase 1 escalation: pass rate and mean tool-invocations per condition. Shows the 35–45% tool-invocation reduction.

`\includegraphics{phase1_cost_passrate.png}`

## 5 Harness Hardening: Three Silent Defects

The extension's most concrete deliverable may be a more correct harness. Three silent defects in the shared runner were found and fixed across the course of this work, each verified before and after against golden references.

**Defect A — simulator working directory.** The simulation invocation set no working directory, so any testbench doing relative-path file I/O (`$readmemh`) could never resolve its data file, *regardless of whether the generated logic was correct.* Disqualifying for every file-I/O benchmark design. Confirmed *not* to affect the conference benchmark.

**Defect B — unclosed code-fence extraction.** An unclosed ```` ```verilog ```` fence left the opening marker attached, corrupting otherwise-compilable code. An audit found 58 of 659 generated files (8.8%) in the Phase 1 benchmark affected.

**Defect C — pass-string mismatch on RTLLM.** The harness checked for the case-sensitive substring `"PASS"` or `"ALL TESTS PASSED"`, but RTLLM's testbenches emit `"Your Design Passed"`—which contains neither. Discovered during the twin pilot analysis (Section 7) when `counter_12`, a module known to be near-trivial, showed 0/3 in the raw data. A full re-scan of all 64 completed runs' simulation logs found 20 (31%) silently misclassified as failures. Worse, this corrupted *trajectories*: a cell that actually passed on iteration 1 was told "SIMULATION FAILED" and spent

its remaining 7 iterations "fixing" already-correct code under a false-failure signal. All results reported in Section 7 use the corrected values, derived from ground-truth simulation logs, not the raw harness output.

That this class of defect recurred three times—each in a different manifestation of the same underlying fragility (a hardcoded pass-string that doesn't match the benchmark's actual output format)—is itself a finding worth reporting. The full HPC study adopts a unified, benchmark-aware pass-detection interface as a direct consequence.

# 6 External Benchmark Migration: RTLLM Pilot

To test generalisation beyond the in-house suite we migrated to RTLLM v2.1 [1], an external 50-design benchmark. A batch compatibility screen found 44 of 50 designs compatible with the Icarus flow; the 6 exclusions each have a distinct, documented cause (one is a defect in RTLLM's own golden reference). A five-module trial (fixed alphabetical-per-category selection, five local models, all STITCH conditions, $N$=3) completed three modules before a thermal stop.

The completed data shows no condition differentiation, for an instructive reason: the three modules span a ceiling (`counter_12`, ∼100% regardless of condition), a floor (`accu`, ∼0%), and a flat mid-point (`barrel_shifter`, identical 20% everywhere). None sits in the band where STITCH's effect is measurable.

Table 5: RTLLM pilot, three completed modules, pooled across five models. **TRIAL — NOT A RESULT.**

| Condition | accu | counter_12 | barrel_shifter |
|---|---|---|---|
| baseline | 1/15 | 15/15 | 3/15 |
| raw_adv | 0/15 | 14/15 | 3/15 |
| sem_adv | 0/15 | 15/15 | 3/15 |
| sem_escalate | 0/15 | 15/15 | 3/15 |
| sem_esc_restart | 0/15 | 15/15 | 3/15 |

The pilot's real yield is a design constraint for the full study: STITCH's effect is observable only on modules in a specific difficulty band—baseline sometimes fails but is recoverable—so the full study must report *per-module* results against a difficulty characterisation rather than a single pooled figure.

# 7 Fine-Tuning Twin Comparison on HPC

## 7.1 Motivation and Design

RTLCoder-Deepseek-v1.1 [2] is fine-tuned on a curated Verilog dataset from the same DeepSeek-coder-6.7b base weights already in our evaluation. This creates a natural experiment: identical base weights, differing only in fine-tuning, so any performance gap isolates the fine-tuning contribution. We additionally include gemma3:4b and llama3.1 as ladder context (smaller and same-scale general-purpose models) and run RTLCoder under *both* its native stub-completion prompt and the shared harness prompt, making prompt format a measured variable rather than a confound.

**Scope**: 5 model-conditions (RTLCoder native, RTLCoder shared, deepseek-coder:6.7b shared, gemma3:4b shared, llama3.1 shared) × 5 modules × $N$=3 = 75 pre-registered runs, all at $T$=0.5 (matched across models for comparability; not comparable to the $T$=0.1 results elsewhere in this

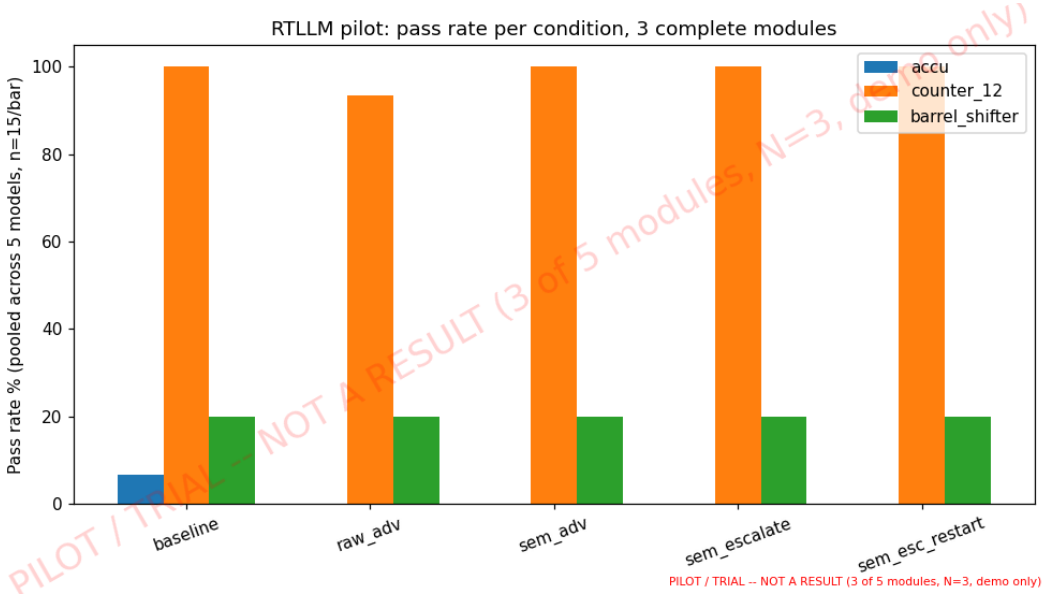



Figure 1: RTLLM pilot pass rate per condition, 3 complete modules (grouped bars, watermarked TRIAL). Shows the ceiling/floor/flat structure the pooled number hides.

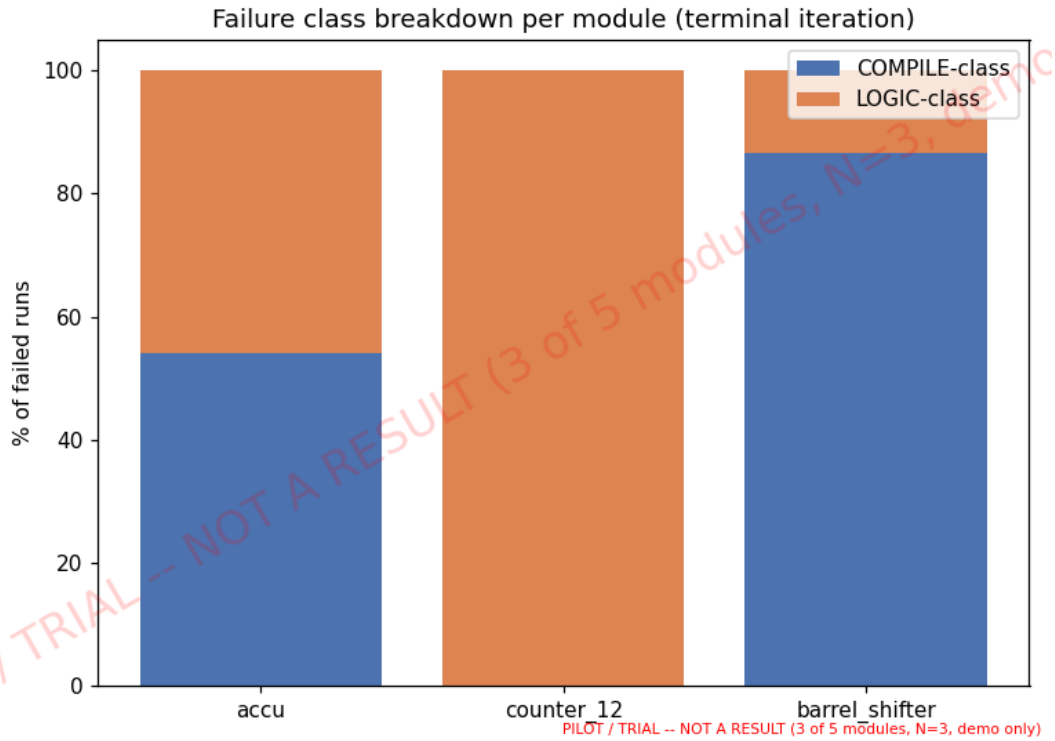



Figure 2: RTLLM pilot failure-class breakdown (COMPILE vs LOGIC) per module.

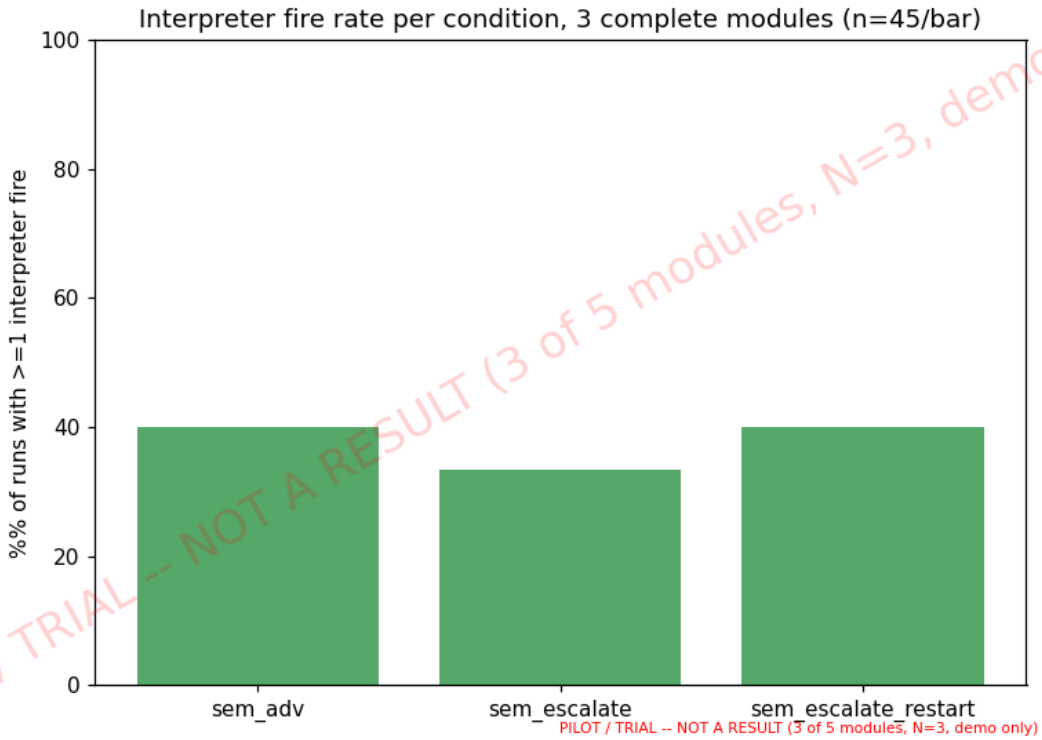



Figure 3: RTLLM pilot interpreter fire-rate per condition.

paper). Condition fixed at `baseline` throughout—this is a *model* comparison, not a STITCH-interpreter evaluation.

64 of 75 runs completed before the HPC wall-clock limit; all 11 missing cells are `calendar` (the last module in run order). Zero errors.

### 7.2 Results

Table 6 presents the corrected per-cell pass rates (see Section 5, Defect C). Four of five modules show zero discriminating power between any model-condition.

Table 6: Twin pilot pass-rate grid (corrected). Gray cells = no data (wall-clock timeout, not zero). **PILOT — 64/75 runs, NOT confirmatory.**

| Module | RTLCoder native | RTLCoder shared | deepseek shared | gemma3 :4b | llama3.1 shared |
|---|---|---|---|---|---|
| accu | 0/3 | 0/3 | 0/3 | 0/3 | 0/3 |
| counter_12 | 3/3 | 3/3 | 3/3 | 3/3 | 3/3 |
| barrel_shifter | 0/3 | 0/3 | **1/3** | 0/3 | 0/3 |
| alu | 0/3 | 0/3 | 0/3 | 0/3 | 0/3 |
| calendar | 3/3 | 1/1† | — | — | — |

†Partial ($n$=1, wall-clock timeout interrupted this cell). Gray cells: no data (timeout), not zero—do not interpret.

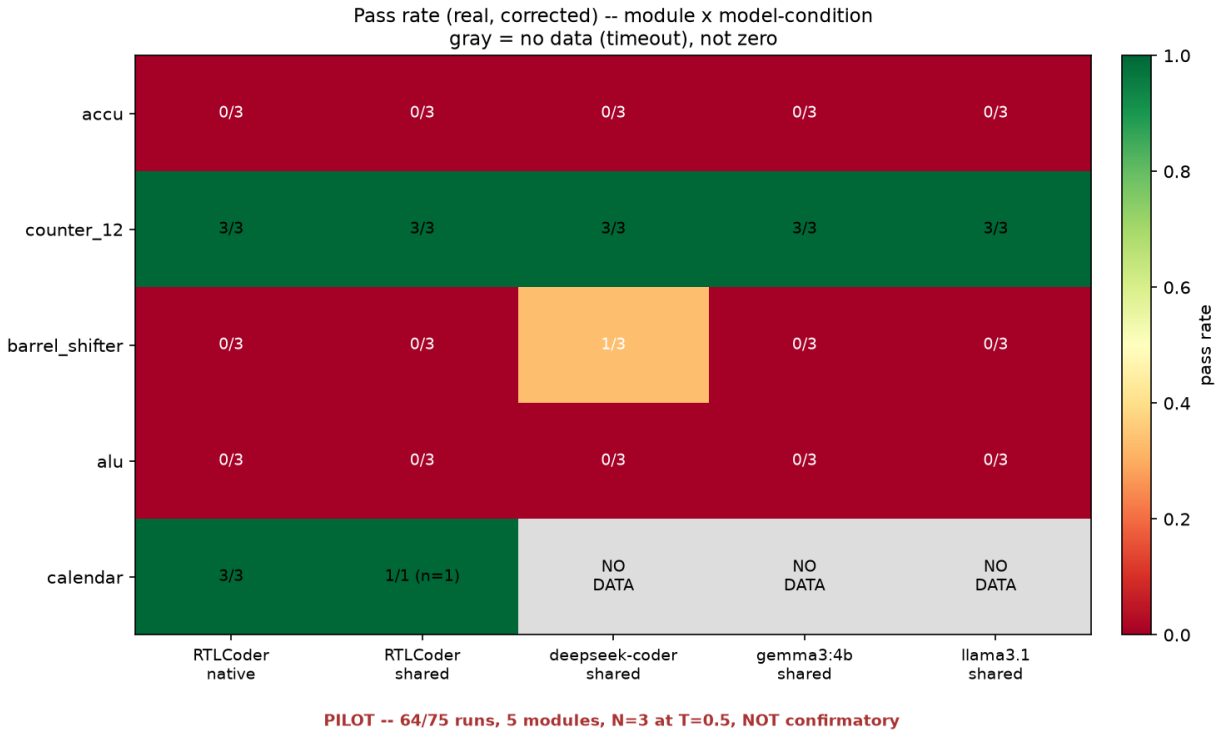



Figure 4: Pass-rate grid, corrected. Gray cells are missing data from the wall-clock timeout, visually distinct from zero. **PILOT — NOT CONFIRMATORY.**

### 7.3 The Two Named Comparisons

**(a) Fine-tuning twin** (RTLCoder/shared vs. deepseek-coder/shared): across the four modules with complete data, three tie exactly (both 0/3 or both 3/3—module ceiling/floor structure, not a model difference) and the one module with any separation (`barrel_shifter`) goes *against* RTLCoder by a single replicate. A one-replicate gap at $N$=3 is not distinguishable from resampling noise. **No fine-tuning advantage is shown in either direction on the completed data.**

**(b) Prompt-format effect** (RTLCoder native vs. RTLCoder shared): zero separation on every module with complete data. Native and shared are indistinguishable in this pilot. `calendar` shows 3/3 vs. 1/1 (partial), which looks consistent but with shared at $n$=1 it cannot be reported as a real data point.

### 7.4 No Capability-Conditional Pattern Observed

We checked for the Phase 1 hypothesis (escalation's value is inversely related to model capability). This pilot's design has no STITCH-interpreter axis (condition fixed at `baseline` throughout), so the hypothesis is not directly testable here. Pooling across the four fully-covered modules (excluding `calendar`, where only RTLCoder has data—pooling it would silently favour RTLCoder as an artefact of the timeout), every model-condition lands at 3/12 or 4/12—flat, no pattern (Figure 6).

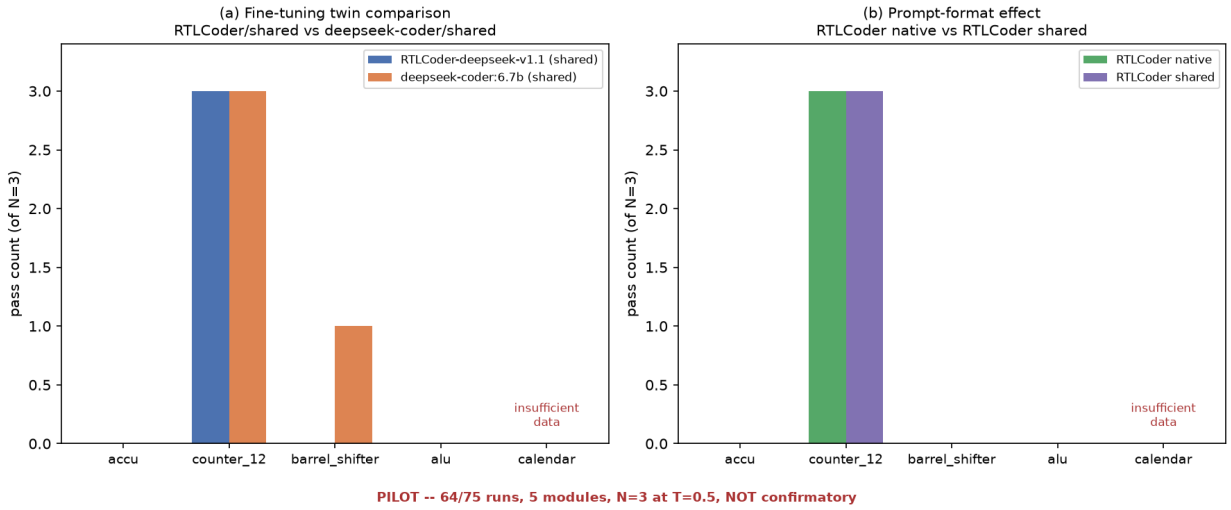



Figure 5: The two named comparisons side by side. `calendar` marked "insufficient data" in both. **PILOT — NOT CONFIRMATORY.**

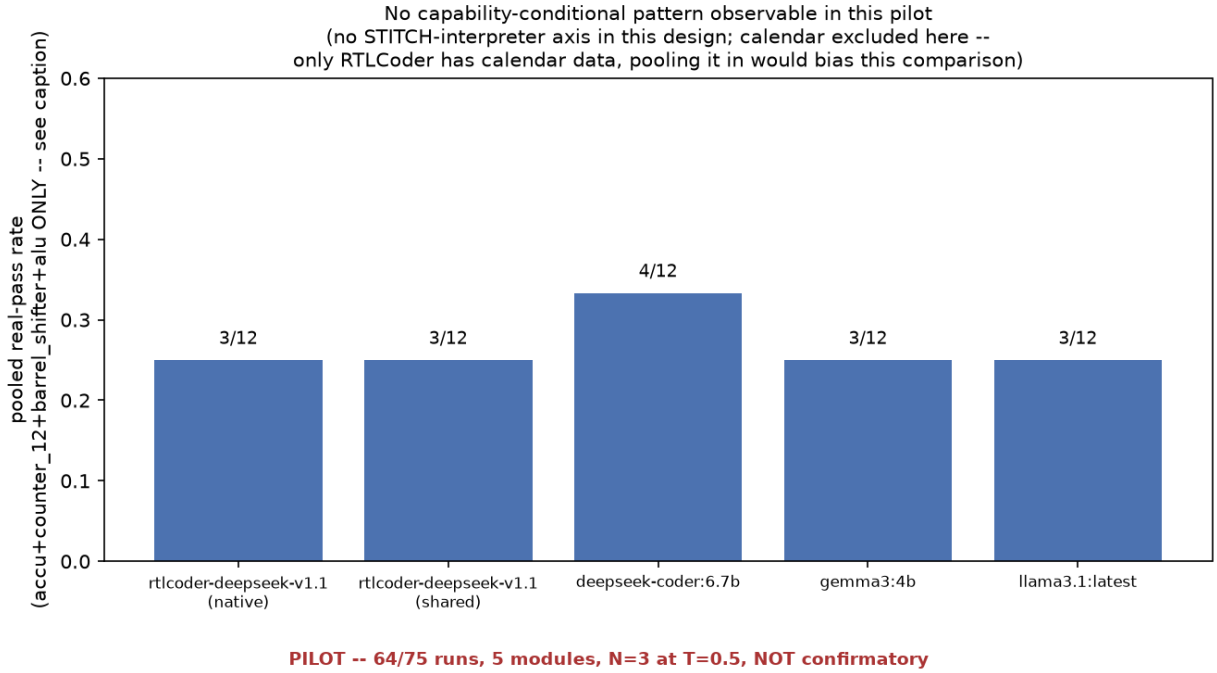



Figure 6: No capability-conditional pattern observable. Calendar excluded to prevent a missing-data artefact. **PILOT — NOT CONFIRMATORY.**

### 7.5 Verdict

This pilot is a correctness check and a first read, not a confirmatory result. Its most concrete finding is Defect C (Section 5)—a pass-detection bug that silently misclassified 31% of completed runs. On the corrected data, 4 of 5 modules show zero discriminating power between any model-condition, for the same structural reason as the Phase 1 RTLLM pilot: the module mix landed almost entirely outside the recoverable band where any comparison could show something either way.

RTLCoder-Deepseek-v1.1 does not outperform its un-fine-tuned twin on the completed data; it does not underperform it either (one single-replicate gap, noise at $N$=3). The prompt-format comparison shows no effect. The one module where RTLCoder shows a real signal (`calendar`, 3/3 native) is precisely the module where every other model has no data due to the timeout. Whether that signal holds once the Ollama models actually reach `calendar` is the open question for the full study—not resolved here and not claimed.

This is a *model* comparison, not a STITCH validation, and stays that way regardless of outcome. Nothing in this pilot's design runs STITCH's interpreter.

## 8 Toward the Full Study

The full study, deferred to HPC, runs the complete 44-design compatible pool at pass@5 across a 4B–32B model ladder with per-module difficulty reporting built in, the RTLCoder twin comparison, and Defect C's fix deployed. Three corrections from the pilots are non-negotiable prerequisites:

1. **Pass-detection**: the shared runner must match RTLLM's actual pass string (`"Your Design Passed"`), verified against a golden reference before any run.
2. **Per-module difficulty**: every result table reports per-module pass rates alongside any aggregate, so ceilings and floors do not drown mid-band signal as they did in both pilots.
3. **Wall-clock budget**: sized off the observed worst-case module cost (alu-class, full-cap-burning), not the mean of easy modules.

Two commitments carry throughout: every interpreter claim is $N \geq 3$-replicated and `semantic_fired`-gated, and every pass-rate figure is reported against the difficulty band that determines whether the mechanism could have been measured. The conference paper showed STITCH *could* help; this extension establishes *when*, *how much*, and *at what cost*.

## References


[1] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model," in *Proc. ASP-DAC*, 2024.

[2] S. Liu, Y. Lu, W. Fang, M. Li, and Z. Xie, "OpenLLM-RTL: Open Dataset and Benchmark for LLM-Aided Design RTL Generation (Invited)," in *Proc. ICCAD*, 2024.